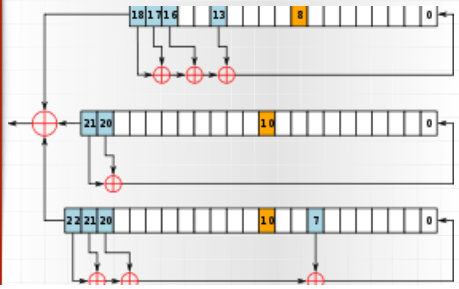
Cybersecurity

Symmetric Ciphers



Stream Ciphers

- Generalization of **one-time pad**
- Stream cipher is initialized with short **key**
- Key is "stretched" into long **keystream**
- Keystream is used like a one-time pad
  - XOR to encrypt or decrypt
- Stream cipher is a keystream generator
- Usually, keystream is bits, sometimes bytes

Stream ciphers

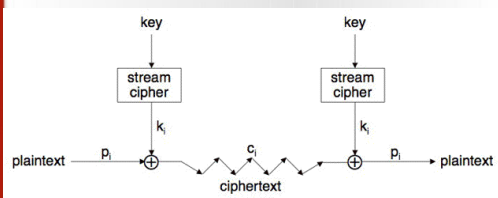
- The CT is generated according to the concept of OTP

M_i xor $K_i = C_i$

Plain Text	0110 1010
Key stream	1100 1011
Cipher text	1010 0001

- We need the generator of key stream
- Generator produces sequences of random values of bytes or bits

Stream Ciphers



Stream Ciphers

- Stream ciphers require random streams of bits
  - True Random Number Generators (TRNGs)
    - Based on physical random processes: coin flipping, dice rolling, semiconductor noise, radioactive decay, clock jitter of digital circuits
    - Output stream should have good statistical properties: $Pr(s_i = 0) = Pr(s_i = 1) = 50\%$
    - Output can neither be predicted nor be reproduced
    - Typically used for generation of keys, nonces (used only-once values) and for many other purposes

Stream Ciphers

- Stream ciphers require random streams of bits
  - Pseudo-random Number Generator (PRNG)
    - Generate sequences from initial seed value
    - Typically, output stream has good statistical properties
    - Output can be reproduced and can be predicted
  - Cryptographically Secure Pseudorandom Number Generator (CSPRNG)
    - Given n consecutive bits of output s_i the following output bits s_{n+1} cannot be predicted (in polynomial time)

Cybersecurity

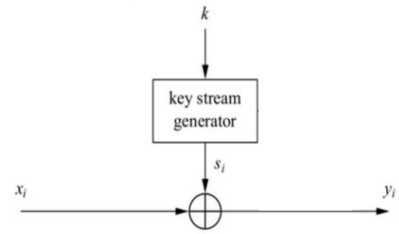
Stream Ciphers

- Two classes of them:
  - Synchronous
  - Asynchronous

Cybersecurity

Stream Ciphers

- Synchronous stream ciphers:



Cybersecurity

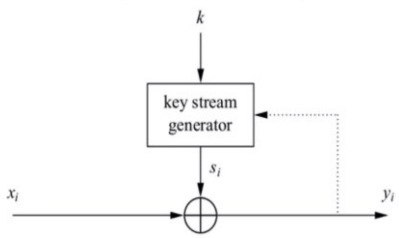
Stream Ciphers

- Synchronous stream ciphers:
  - Key is independent of plaintext and of ciphertext
  - Easy to generate
  - No error propagation
  - Insertion, deletion can be detected
  - Synchronization required
  - Data authentication and integrity required
  - Sequence needs to be „strong"

Cybersecurity

Stream Ciphers

- Asynchronous



Cybersecurity

Stream Ciphers

- Asynchronous
  - Stream cipher is dependent on *t* previous ciphertext digits
  - Self-synchronized and limited error propagation
  - More difficult than synchronous ciphers with respect to detection of insertion and deletion
  - Plaintext statistics are dispersed throughout the ciphertext.
  - More resistant to eavesdropping
  - Harder to generate

Cybersecurity

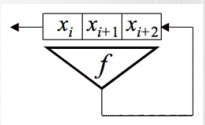
Shift Registers

- Traditionally, stream ciphers were based on shift registers
  - Today, a wider variety of designs
- Shift register includes
  - A series of stages each holding one bit
  - A feedback function
- A Linear Feedback Shift Register (**LFSR**) has a linear feedback function

Cybersecurity

Shift Registers

- Example (nonlinear) feedback function
  $f(x_i, x_{i+1}, x_{i+2}) = 1 \oplus x_i \oplus x_{i+2} \oplus x_{i+1}x_{i+2}$
- Example (nonlinear) shift register

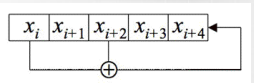


- First 3 bits are **initial fill**: (x_0, x_1, x_2)

Cybersecurity

LFSR

- For LFSR



- We have $x_{i+5} = x_i \oplus x_{i+2}$ for all i
- Linear feedback functions often written in polynomial form: $x^5 + x^2 + 1$
- **Connection polynomial** of the LFSR

Cybersecurity

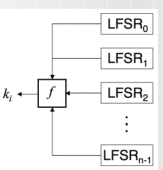
Shift Register-Based Stream Ciphers

- Two approaches to LFSR-based stream ciphers
  - One LFSR with nonlinear combining function
  - Multiple LFSRs combined via nonlinear function
- In either case
  - Key is initial fill of LFSRs
  - Keystream is output of nonlinear combining function

Cybersecurity

Shift Register-Based Stream Ciphers

- LFSR-based stream cipher
  - Multiple LFSRs with nonlinear function
  - Keystream: $k_0, k_1, k_2, \ldots$



Cybersecurity

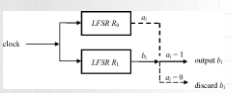
Shift Register-Based Stream Ciphers

- Single LFSR example is special case of multiple LFSR example
- To convert single LFSR case to multiple
  - Let $LFSR_0, \ldots LFSR_{n-1}$ be same as LFSR
  - Initial fill of $LFSR_0$ is initial fill of LFSR
  - Initial fill of $LFSR_1$ is initial fill of LFSR stepped once
  - And so on…

Cybersecurity

Example LFSR-Based Ciphers

- The Shrinking Generator:
  - Two LFSRs
  - If $x_{(1)}$ is 1, output $x_{(2)}$.
    Else, discard both $x_{(1)}$ & $x_{(2)}$; forward the LFSRs.

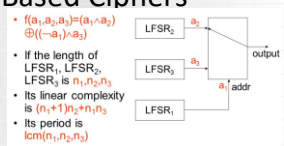


- A5 (the GSM standard):
  - Three LFSRs; 64 bits in total.
  - Designed secretly. Leaked in 1994.
  - Completely broken. (Biryukov & Shamir, 1999)
- E0 (Bluetooth's standard encryption)
  - Four LFSRs; 128 bits in total.

Cybersecurity

Example LFSR-Based Ciphers

- Geffe Generator:



Correlation problem:
$P(y = x_{(2)}) = 0.75$
(or, $P(y = x_{(3)})$)

- Stop-and-Go Generators:
  - One (or more) LFSR is used to clock the others
  - E.g.: The alternating stop-and-go generator: Three LFSRs. If $x_{(1)}$ is 0, $LFSR_2$ is forwarded; otherwise $LFSR_3$. Output is $x_{(2)} \oplus x_{(3)}$.

Cybersecuirty

Blum-Micali Generator

- Based on the discrete logarithm one-way function
- Let p be an odd prime
- Let a be a generator of the group
- Let x_0 be a seed

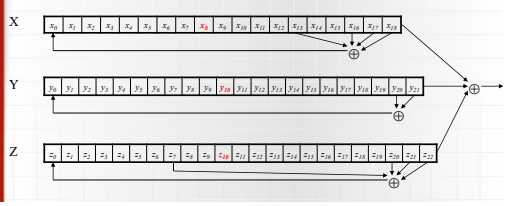
$$x_{i+1} = a^{x_i} \pmod p \text{ dla } i = 1, 2, 3, \ldots$$

- Output
  - $y_i=1$ if $x_i>=(p-1)/2$
  - $y_i=0$ otherwise
  - $Y=(y_1,y_2,\ldots,y_k)$ <-pseudo-random sequence of K bits

Cybersecurity

A5/1

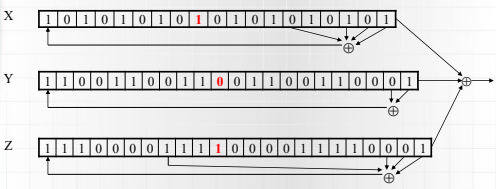
- Each value is a single bit
- Key is used as **initial fill** of registers
- Each register steps or not, based on (x_8, y_{10}, z_{10})
- Keystream bit is XOR of right bits of registers



Cybersecurity

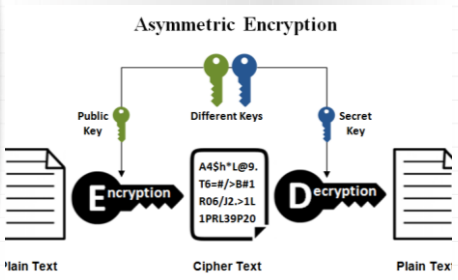
A5/1

- In this example, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(1,0,1) = 1$
- Register X steps, Y does not step, and Z steps
- Keystream bit is XOR of right bits of registers
- Here, keystream bit will be $0 \oplus 1 \oplus 0 = 1$



Cybersecurity

Asymetric Ciphers



Cybersecurity

Agenda

- We briefly discuss the following
  - Merkle-Hellman knapsack
  - Diffie-Hellman key exchange
  - RSA
  - Rabin Cipher
  - Digital Signature

Cybersecurity

Public Key Crypto

- What can not we do using only symetric ciphers?
- Some public key systems provide encryption, digital signatures, etc.
  - For example, RSA
- Some are only for key exchange
  - For example, Diffie-Hellman
- Some are only for signatures
  - For example, ElGamal
- All of these are public key systems

Cybersecurity

Merkle-Hellman Knapsack

- One of first public key systems
- Based on NP-complete problem
- Original algorithm is weak
  - Lattice reduction attack
- Newer knapsacks are more secure
  - But nobody uses them…

Cybersecurity

Knapsack Problem

- Given a set of n weights $W_0, W_1, \ldots, W_{n-1}$ and a sum S, is it possible to find $a_i \in \{0,1\}$ so that

  $$S = a_0W_0 + a_1W_1 + \ldots + a_{n-1}W_{n-1}$$

  (technically, this is "subset sum" problem)
- **Example**
  - Weights (62,93,26,52,166,48,91,141)
  - Problem: Find subset that sums to S = 302
  - Answer: 62+26+166+48 = 302
- The (general) knapsack is NP-complete

Cybersecurity

Knapsack Problem

- General knapsack (GK) is hard to solve
- But **superincreasing knapsack** (SIK) is easy
- In SIK each weight greater than the sum of all previous weights
- **Example**
  - Weights (2,3,7,14,30,57,120,251)
  - Problem: Find subset that sums to S = 186
  - Work from largest to smallest weight
  - Answer: 120+57+7+2 = 186

Cybersecurity

Knapsack Cryptosystem

1. Generate superincreasing knapsack (SIK)
2. Convert SIK into "general" knapsack (GK)
3. **Public Key:** GK
4. **Private Key:** SIK plus conversion factors

- Easy to encrypt with GK
- With private key, easy to decrypt (convert ciphertext to SIK)
- Without private key, must solve GK ?

Cybersecurity

Knapsack Cryptosystem

- Let (2,3,7,14,30,57,120,251) be the SIK
- Choose m = 41 and n = 491
  - m and n relatively prime,
  - n > sum of SIK elements
- General knapsack

  $2 \cdot 41 \pmod{491} = 82$
  $3 \cdot 41 \pmod{491} = 123$
  $7 \cdot 41 \pmod{491} = 287$
  $14 \cdot 41 \pmod{491} = 83$
  $30 \cdot 41 \pmod{491} = 248$
  $57 \cdot 41 \pmod{491} = 373$
  $120 \cdot 41 \pmod{491} = 10$
  $251 \cdot 41 \pmod{491} = 471$
- General knapsack: (82,123,287,83,248,373,10,471)

Cybersecurity

Knapsack Cryptosystem

- **Private key:** (2,3,7,14,30,57,120,251)
  $m^{-1} \bmod n = 41^{-1} \pmod{491} = 12$
- **Public key:** (82,123,287,83,248,373,10,471), n=491
- Example: Encrypt 10010110
  $82 + 83 + 373 + 10 = 548$
- To decrypt,
  - $548 \cdot 12 = 193 \pmod{491}$
  - Solve (easy) SIK with S = 193
  - Obtain plaintext 10010110

Cybersecurity

Knapsack Cryptosystem

- **Trapdoor:**
  - Convert SIK into "general" knapsack using modular arithmetic
- **One-way:**
  - General knapsack easy to encrypt,
  - hard to solve;
  - SIK easy to solve
- This knapsack cryptosystem is **insecure**
  - Broken in 1983 with Apple II computer
  - The attack uses **lattice reduction**

Cybersecurity

Knapsack Cryptosystem

- Lattice reduction is a surprising method of attack on knapsack
- A cryptosystem is only secure as long as nobody has found an attack
- Lesson:
  Advances in mathematics can break cryptosystems

Cybersecurity

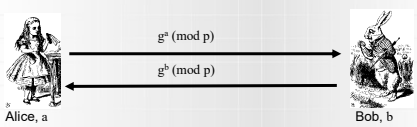
Diffie-Hellman Key Exchange

- Invented by Williamson and, independently, by Diffie and Hellman (Stanford)
- A "key exchange" algorithm
  - To establish a shared symmetric key
- Not for encrypting or signing
- Security rests on difficulty of **discrete log** problem: given g, p, and $g^k \pmod p$, find k

Cybersecurity

Diffie-Hellman Key Exchange

- **Public:** g and p
- **Secret:** Alice's exponent a, Bob's exponent b



- Alice computes $(g^b)^a = g^{ba} = g^{ab} \pmod p$
- Bob computes $(g^a)^b = g^{ab} \pmod p$
- Could use $K = g^{ab} \pmod p$ as symmetric key

Cybersecurity

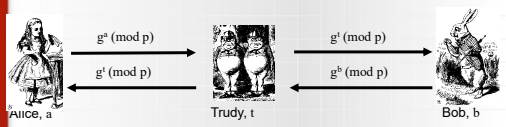
Diffie-Hellman Key Exchange

- Suppose that Bob and Alice use $g^{ab} \pmod p$ as a symmetric key
- Trudy can see $g^a \pmod p$ and $g^b \pmod p$
- Note $g^a g^b = g^{a+b} \neq g^{ab} \pmod p$
- If Trudy can find a or b, system is broken
- If Trudy can solve **discrete log** problem, then she can find a or b

Cybersecurity

Diffie-Hellman Key Exchange

- Subject to man-in-the-middle (MiM) attack



- Trudy shares secret $g^{at} \pmod p$ with Alice
- Trudy shares secret $g^{bt} \pmod p$ with Bob
- Alice and Bob don't know Trudy exists!

Cybersecurity

Diffie-Hellman Key Exchange

- How to prevent MiM attack?
  - Encrypt DH exchange with symmetric key
  - Encrypt DH exchange with public key
  - Sign DH values with private key
  - Other?
- You **MUST** be aware of MiM attack on Diffie-Hellman

Cybersecurity